

Parte 4

1. Diferencia entre Git y GitHub

- Pregunta: Explica la diferencia entre Git y GitHub.
- Respuesta: Git es el sistema o software de control de versiones que se ejecuta en tu computadora local para rastrear el historial de cambios en tus archivos. GitHub es una plataforma en la nube diseñada para alojar los repositorios de Git, permitiendo respaldar tu código en internet y facilitar el trabajo colaborativo con otras personas.

2. Función de .gitignore

- Pregunta: Explica para qué sirve .gitignore.
- Respuesta: Es un archivo de texto donde se enlistan las carpetas y archivos que Git debe ignorar por completo. Su función es evitar que archivos basura, cachés temporales, información sensible o carpetas muy pesadas se incluyan en el control de versiones y se suban a GitHub.

3. Exclusión de .venv

- Pregunta: Explica por qué .venv no debe almacenarse normalmente en GitHub.
- Respuesta: La carpeta .venv contiene cientos de archivos ejecutables y bibliotecas instaladas que son muy pesados y sobre todo específicos del sistema operativo local. Subirla generaría conflictos si otro desarrollador usa un sistema distinto. Es más eficiente que cada persona genere su propio entorno virtual en su máquina.

4. Función de requirements.txt

- Pregunta: Explica para qué sirve requirements.txt.
- Respuesta: Sirve como un inventario o lista de compras. Contiene los nombres y las versiones exactas de las librerías o dependencias externas que el proyecto necesita para funcionar. Al compartir este archivo cualquier otro desarrollador puede instalar todo el entorno de trabajo rápidamente usando un solo comando.

5. Diferencia entre Stage, Commit y Push

- Pregunta: Explica la diferencia entre Stage, Commit y Push.
- Respuesta:
 - Stage Es la sala de espera. Seleccionas qué archivos modificados quieres incluir en el próximo guardado

- Commit Es tomar la fotografía permanente de los archivos que estaban en el Stage. Guarda el registro de los cambios en el historial de tu computadora
- Push (Empujar/Enviar): Es el acto de tomar todos esos Commits que guardaste localmente y subirlos al servidor remoto en internet, como GitHub (se hace con git push).

6. Varios commits antes de un push

- Pregunta: Explica por qué un repositorio puede tener varios commits antes de realizar un push.
- Respuesta: Porque Git funciona de manera distribuida y local. Un desarrollador puede trabajar sin conexión a internet y hacer múltiples "commits" a lo largo del día para registrar su progreso y guardar hitos lógicos paso a paso en su máquina. Una vez que termina su bloque de trabajo o recupera la conexión, utiliza un solo "push" para enviar de golpe todos esos commits acumulados al servidor remoto.